## Recursion with Pattern Matching

Recursion is the most primitive form of iteration and reduction in Elixir. A recursive function is one that calls itself. If a function can call itself, it needs some idea of when to stop. This is called the base case, and every recursive function needs one, or it will execute infinitely.

Some problems are most easy to solve when thought about recursively. Because of this, you'll often see recursion appear in libraries or even in your own code. It's a little scary at first, but you'll appreciate its value as you get comfortable with it.

Pattern matching goes hand in hand with recursion because you can build the base case using pattern matching. The most practical example of this is list iteration. Create lib/examples/patterns/recursion.ex with the following code:

elixir_examples/lib/examples/patterns/recursion.ex

```
Line 1  defmodule Examples.Patterns.Recursion do
        def biggest_number(list) do
          biggest_number(list, nil)
        end
      5
        defp biggest_number([], max), do: max
        
        defp biggest_number([head | tail], current_max) do
          next_max = max(head, current_max || head)
     10   biggest_number(tail, next_max)
        end
      end
```

```
$ iex -S mix
iex> Examples.Patterns.Recursion.biggest_number([2, 1, 5, 1])
5

iex> Examples.Patterns.Recursion.biggest_number([])
nil
```

Line 2 is a public function definition that accepts a single argument. This allows biggest_number/1 to be called without worrying about the accumulator (current_max) being set.

Line 6 is the base case for when the input list is empty. This function relies on pattern matching to execute only when the list is empty. It returns the accumulated biggest_number, which gets returned back to the function caller.

Line 8 is the main body of the recursive function. The argument list uses pattern matching to grab the first element of the input list (head) and the rest of the input list (tail). The function then calculates the maximum value and recursively invokes itself.